

Client-Centric Consistency in a Replicated MongoDB Database

Experiments under replica lag, node failure, and network partition

Course	DSA5208 Distributed Systems
Database	MongoDB Community 7.0.32
Deployment	Three Docker containers, one replica set
Experiment date	22 September 2026
Group	<i>Add group members and student IDs before submission</i>

Submission note. The report is generated from an audited formal run containing 640 client histories and 2720 attempted operations. The accompanying archive includes the raw histories, three server logs, metadata, checksums, and reproduction scripts.

Abstract

This project evaluates four client-centric consistency properties—read-your-writes, monotonic reads, monotonic writes, and writes-follow-reads—in a three-member MongoDB replica set. Eight client configurations form a complete $2 \times 2 \times 2$ matrix over causal history tracking, read concern, and write concern. The same application history is exercised under seven equally important operating scenarios: normal operation, deliberate replica lag, secondary failure, primary failure, a 2+1 network partition, complete 1+1+1 isolation, and partition-induced rollback. The formal run contains 640 histories and 2720 attempted operations. Normal operation and single-node failures established the healthy and failover baselines. Replica lag and both partition patterns exposed the difference between stale successful reads and operations that wait or fail to preserve causal order. The rollback scenario exposed the durability consequences of acknowledged $w : 1$ writes. Across the full matrix, the observations agree with the predicted client-centric guarantees and demonstrate that consistency and availability must be interpreted together.

Contents

1	System and deployment	2
1.1	Chosen database	2
1.2	Architecture	2
1.3	Installation and startup	2
2	Consistency configurations and predictions	2
3	Experimental method	3
3.1	Workload and oracle	3
3.2	Scenarios	4
4	Results	4
4.1	Normal operation	5
4.2	Replica lag	5
4.3	Secondary crash	6
4.4	Primary crash and election	6
4.5	2+1 network partition	6
4.6	1+1+1 complete isolation	7
4.7	Partition followed by rollback	7
5	Comparison with predictions	8
6	Limitations	8
7	Reproducibility and submitted evidence	8
8	AI usage disclosure	9
A	Exact configuration summary	9

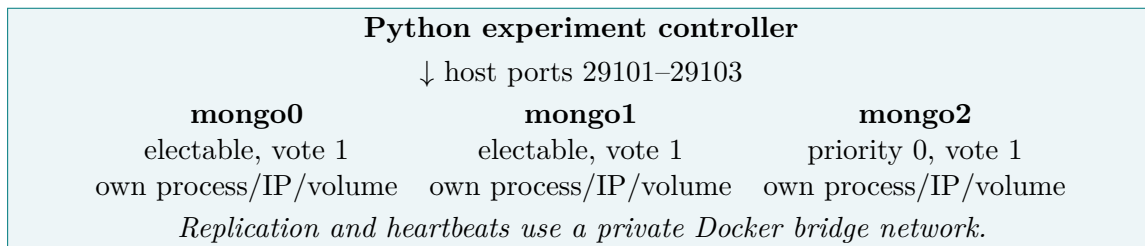
1 System and deployment

1.1 Chosen database

MongoDB is a document database whose replica sets provide automatic primary election, asynchronous replication, configurable read and write concerns, and causally consistent client sessions. These mechanisms make it possible to vary consistency and durability on a per-operation basis while preserving the same application workload [1, 2, 3].

The experiment uses MongoDB Community 7.0.32 and PyMongo 4.10.1. All versions are pinned in the Dockerfile and `requirements.txt`. The storage engine is WiredTiger.

1.2 Architecture



Each container stores a complete data copy in a named volume. All three members vote, but `mongo2` has priority zero so that it cannot become primary. Nodes 0 and 1 are electable. The replica set therefore survives one node failure and can elect a primary on the two-node side of a 2+1 split. The controller uses direct connections so that each read target is known. Causal metadata is explicitly transferred when the logical client changes physical connections; a PyMongo session is never passed to a different `MongoClient`.

This is a distributed logical deployment using multiple containers on one physical Mac, as permitted by the assignment. It does not emulate independent power, disks, or host kernels.

1.3 Installation and startup

Docker Desktop and Docker Compose are required. From the repository root:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m unittest -v test_checker.py
.venv/bin/python run.py --work-dir work/matrix8-timeout5s-full-02 \
--out results/matrix8-timeout5s-full-02
.venv/bin/python analyze.py results/matrix8-timeout5s-full-02
```

The runner builds the pinned MongoDB image, creates fresh containers and volumes, calls the replica-set initialization command, waits for one primary and two secondaries, performs the tests, heals all partitions, collects logs, and stops the containers. New directories are required for every run, preventing accidental reuse of earlier data.

2 Consistency configurations and predictions

MongoDB read concern specifies which data a read may expose, while write concern specifies how many voting members must acknowledge a write. A causally consistent session carries logical time from an earlier operation to a later operation. For reads, the driver sends an `afterClusterTime` lower bound so that a server cannot return an older snapshot merely to respond quickly [1, 7].

Table 1: Client configurations.

ID	Causal tracking	Read concern	Write concern
A	off	local	$w : 1$
B	on	local	$w : 1$
C	off	local	$w : \text{majority}$
D	on	local	$w : \text{majority}$
E	off	majority	$w : 1$
F	on	majority	$w : 1$
G	off	majority	$w : \text{majority}$
H	on	majority	$w : \text{majority}$

The table is ordered as four adjacent matched pairs: A/B, C/D, E/F, and G/H. Within each pair, read concern and write concern are fixed and only causal tracking is changed. This makes the causal-session effect identifiable without changing the other two client settings at the same time.

Table 2 records the predictions made from the documented causal-session matrix. G means a documented guarantee for successful operations under the stated settings; NG means that a permitted execution can violate the property. An NG prediction does not mean every execution must show a violation.

Table 2: Predicted client-centric guarantees.

Configuration	RYW	MR	MW	WFR
A: local/ $w : 1$, causal off	NG	NG	NG	NG
B: local/ $w : 1$, causal on	NG	NG	NG	NG
C: local/majority, causal off	NG	NG	NG	NG
D: local/majority, causal on	G	NG	G	NG
E: majority/ $w : 1$, causal off	NG	NG	NG	NG
F: majority/ $w : 1$, causal on	NG	G	NG	G
G: majority/majority, causal off	NG	NG	NG	NG
H: majority/majority, causal on	G	G	G	G

The report uses a durable-history interpretation: if a write returned an acknowledgment, later loss of that write during rollback can witness an RYW or MW violation. MongoDB also documents a weaker interpretation in which rollback is itself part of the history. We state the chosen interpretation to avoid mixing the two definitions [1, 6].

3 Experimental method

3.1 Workload and oracle

Every trial uses a unique document initialized on all three members as $\{a:0, b:0\}$. The usual operation order is:

W1: set $a=1 \rightarrow R1 \rightarrow R2 \rightarrow W2$: set $b=1$ and return pre-image.

`findOneAndUpdate(..., returnDocument=BEFORE)` makes W2 a real modifying write while atomically revealing the database state on which it executed. This avoids incorrectly inferring write ordering from a later final value.

Table 3: Operational checks for the four properties.

Property	Eligible check	Violation witness
Read-your-writes	W1 acknowledged and at least one later read returned	Any successful read invoked after W1 completed returns $a < 1$
Monotonic reads	R1 and R2 both returned	$R2.a < R1.a$
Monotonic writes	W1 and W2 acknowledged	W2 pre-image has $a < 1$
Writes-follow-reads	W2 acknowledged after a successful read	W2 pre-image is older than the maximum version observed by earlier reads

Read errors expose no value and therefore produce an inconclusive read-to-read check. A write timeout can have an unknown outcome and is not treated as an acknowledged predecessor. Errors and timeouts are reported separately from stale-value violations. Eight synthetic unit tests validate preserved histories, stale reads, timeout handling, rollback, WFR order, and the requirement that RYW examine every successful read following W1.

3.2 Scenarios

Table 4: Operating scenarios and repetitions per configuration.

Scenario	Fault and routing	n
Normal	No injected fault; R2 targets a known secondary.	20
Replica lag	mongo2 applies operations with a three-second delay; R2 targets it.	10
Secondary crash	Kill mongo2 with SIGKILL; the surviving pair remains available.	10
Primary crash	W1/R1, kill primary, await election, then R2/W2 on the new primary.	10
2+1 partition	Isolate the old primary; the other two nodes elect a new primary. W1/R1/W2 use the majority side and R2 probes the minority.	10
1+1+1 isolation	Drop all peer traffic while host-to-container client access remains.	10
Partition + rollback	Isolate before W1, kill old primary, heal survivors, elect a new primary, then execute W2/R2.	10

Faults are implemented with container-local `iptables` INPUT/OUTPUT rules using `NET_ADMIN`. Rules block only peer container addresses; the controller retains access through published loopback ports. Every disruptive trial receives its own cut/heal cycle and an administrative $w : 3$ recovery barrier. The audit validates actual wire commands, read/write concerns, causal bounds, unique trial IDs, and the expected topology of every partition snapshot.

4 Results

The formal run completed 640 histories and 2720 attempted operations. Command monitoring captured 2700 wire commands; 20 rollback operations were rejected by PyMongo before any command was sent because a restarted direct connection temporarily reported that sessions were unsupported.

The audit records these separately and found no missing wire evidence for any successful operation, no incorrect concern parameters, and no failure among 160 isolated topology snapshots. In every table, RC and WC denote read and write concern, and property cells are *violations/eligible checks*. “–” means a prerequisite did not return, so the property is inconclusive. Errors are availability outcomes, not consistency violations.

4.1 Normal operation

Table 5: Normal operation: violations/eligible checks and errors/operations.

Config	Causal	RC	WC	RYW	MR	MW	WFR	Errors
A	off	local	1	0/20	0/20	0/20	0/20	0/80
B	on	local	1	0/20	0/20	0/20	0/20	0/80
C	off	local	majority	0/20	0/20	0/20	0/20	0/80
D	on	local	majority	0/20	0/20	0/20	0/20	0/80
E	off	majority	1	20/20	0/20	0/20	0/20	0/80
F	on	majority	1	0/20	0/20	0/20	0/20	0/80
G	off	majority	majority	0/20	0/20	0/20	0/20	0/80
H	on	majority	majority	0/20	0/20	0/20	0/20	0/80

All nodes were healthy and R2 targeted `mongo2`. All 640 operations succeeded, so the error column is zero. E produced 20 RYW witnesses: $w : 1$ acknowledged W1 before it became majority committed, while its non-causal majority read could return the older committed snapshot. F used the same concerns but carried causal time and returned no stale successful read. Thus majority read concern alone is not RYW. The other zero cells are observations, not universal proofs.

4.2 Replica lag

Table 6: Three-second replica lag: violations/eligible checks and errors/operations.

Config	Causal	RC	WC	RYW	MR	MW	WFR	Errors
A	off	local	1	10/10	10/10	0/10	0/10	0/40
B	on	local	1	0/10	0/10	0/10	0/10	0/40
C	off	local	majority	10/10	10/10	0/10	0/10	0/40
D	on	local	majority	0/10	0/10	0/10	0/10	0/40
E	off	majority	1	10/10	0/10	0/10	0/10	0/40
F	on	majority	1	0/10	0/10	0/10	0/10	0/40
G	off	majority	majority	10/10	10/10	0/10	0/10	0/40
H	on	majority	majority	0/10	0/10	0/10	0/10	0/40

`mongo2` applied oplog entries three seconds late, while reads could wait five seconds. Non-causal A, C, E, and G returned stale values and violated RYW. A, C, and G regressed from R1’s new value to R2’s old value and also violated MR. E read the same older majority snapshot twice, so it violated RYW but not MR: remaining stale is not moving backward. Causal B, D, F, and H attached `afterClusterTime`; unlike the former 700 ms run, they waited for the finite lag to clear and returned current values. Consequently all 320 operations succeeded and no error occurred. This isolates the effect of the new deadline: causal tracking traded about three seconds of latency for a successful consistent result rather than a timeout.

4.3 Secondary crash

Table 7: Secondary crash: violations/eligible checks and errors/operations.

Config	Causal	RC	WC	RYW	MR	MW	WFR	Errors
A	off	local	1	0/10	0/10	0/10	0/10	0/40
B	on	local	1	0/10	0/10	0/10	0/10	0/40
C	off	local	majority	0/10	0/10	0/10	0/10	0/40
D	on	local	majority	0/10	0/10	0/10	0/10	0/40
E	off	majority	1	10/10	0/10	0/10	0/10	0/40
F	on	majority	1	0/10	0/10	0/10	0/10	0/40
G	off	majority	majority	0/10	0/10	0/10	0/10	0/40
H	on	majority	majority	0/10	0/10	0/10	0/10	0/40

mongo2 was killed with SIGKILL; the primary and the remaining secondary retained a majority, and R2 used the surviving secondary. All 320 operations succeeded, so there were no selection, read, or write errors. E again exposed the gap between a $w : 1$ acknowledgment and a majority snapshot, yielding 10 RYW witnesses without MR regression; causal F did not. The surviving two voters explain both continued majority-write availability and the zero errors.

4.4 Primary crash and election

Table 8: Primary crash and election: violations/eligible checks and errors/operations.

Config	Causal	RC	WC	RYW	MR	MW	WFR	Errors
A	off	local	1	0/10	0/10	0/10	0/10	0/40
B	on	local	1	0/10	0/10	0/10	0/10	0/40
C	off	local	majority	0/10	0/10	0/10	0/10	0/40
D	on	local	majority	0/10	0/10	0/10	0/10	0/40
E	off	majority	1	10/10	0/10	0/10	0/10	0/40
F	on	majority	1	0/10	0/10	0/10	0/10	0/40
G	off	majority	majority	0/10	0/10	0/10	0/10	0/40
H	on	majority	majority	0/10	0/10	0/10	0/10	0/40

W1 and R1 ran before SIGKILL; R2 and W2 ran after the surviving nodes elected a new primary. The harness waited for that election before continuing, so all 320 operations succeeded and there were no election-time client errors. E produced RYW witnesses in all 10 histories via the same $w : 1$ /majority-snapshot mismatch; the other configurations did not. This is healthy failover, not proof that every $w : 1$ write survives: replication was not blocked first. The rollback scenario deliberately tests the harder case.

4.5 2+1 network partition

Table 9: 2+1 partition: violations/eligible checks and errors/operations.

Config	Causal	RC	WC	RYW	MR	MW	WFR	Errors
A	off	local	1	10/10	10/10	0/10	0/10	10/50
B	on	local	1	0/10	–	0/10	0/10	20/50
C	off	local	majority	10/10	10/10	0/10	0/10	10/50
D	on	local	majority	0/10	–	0/10	0/10	20/50
E	off	majority	1	10/10	0/10	0/10	0/10	10/50
F	on	majority	1	0/10	–	0/10	0/10	20/50
G	off	majority	majority	10/10	10/10	0/10	0/10	10/50
H	on	majority	majority	0/10	–	0/10	0/10	20/50

The old primary was isolated while the two-node side retained quorum and elected a new primary. W1, R1, and W2 succeeded on that majority side. R2 probed the stale minority. Non-causal A, C,

and G regressed and violated RYW/MR; E violated RYW but not MR because its majority reads stayed on the older committed snapshot. Causal B, D, F, and H refused to return below their causal bound, making MR inconclusive. Exactly 80 errors were deliberate `W_unavailable` probes rejected with `NotPrimaryError`, one per history. The 40 additional errors were causal R2 reads (B, D, F, H) reaching the five-second `ExecutionTimeout`; a continuing partition cannot clear like finite lag. Thus non-causal rows have 10/50 errors and causal rows 20/50.

4.6 1+1+1 complete isolation

Table 10: 1+1+1 isolation: violations/eligible checks and errors/operations.

Config	Causal	RC	WC	RYW	MR	MW	WFR	Errors
A	off	local	1	10/10	10/10	0/10	0/10	10/50
B	on	local	1	0/10	–	0/10	0/10	20/50
C	off	local	majority	–	10/10	–	–	30/50
D	on	local	majority	–	–	–	–	40/50
E	off	majority	1	10/10	0/10	0/10	0/10	10/50
F	on	majority	1	–	–	0/10	–	30/50
G	off	majority	majority	–	0/10	–	–	30/50
H	on	majority	majority	–	–	–	–	50/50

All peer paths were blocked while host-to-container access remained. No two voters could form a majority. A and E could briefly complete $w : 1$ work while the former primary detected quorum loss; A returned a regressing local read, while E returned an older majority snapshot. C returned enough local reads to witness MR regression, but its majority writes failed. Across all rows, 40 W1 and 30 W2 majority writes reached `WTimeoutError`; H's other 10 W2 attempts reached `NotPrimaryError` after step-down. Twenty causal R1 and 40 causal R2 reads reached `ExecutionTimeout`. Finally, all 80 post-step-down write probes were rejected with `NotPrimaryError`. These exact components explain the row totals from 10/50 to H's 50/50. Dashes mean refusal or timeout removed a prerequisite, not a hidden pass.

4.7 Partition followed by rollback

Table 11: Partition followed by rollback: violations/eligible checks and errors/operations.

Config	Causal	RC	WC	RYW	MR	MW	WFR	Errors
A	off	local	1	10/10	10/10	10/10	10/10	0/40
B	on	local	1	6/7	6/6	6/6	6/6	8/40
C	off	local	majority	–	9/9	–	7/7	14/40
D	on	local	majority	–	9/9	–	3/3	18/40
E	off	majority	1	9/9	0/8	8/8	0/8	4/40
F	on	majority	1	7/7	–	9/9	–	14/40
G	off	majority	majority	–	0/10	–	0/6	14/40
H	on	majority	majority	–	–	–	–	34/40

The nodes were isolated before W1. A $w : 1$ write could therefore be acknowledged only on the old primary, which was killed before the other two nodes elected a branch without W1. The W2 pre-image exposed loss of that predecessor. A always completed the unsafe branch and violated all four properties. Timing now matters: B produced six witnesses among seven eligible RYW checks, while E/F also exposed rolled-back $w : 1$ predecessors whenever prerequisites returned. Majority-write C/D/G/H never acknowledged isolated W1, so RYW and MW were inconclusive.

The 106 errors have four measured causes. Forty W1 majority writes and 18 W2 majority writes reached the five-second `WTimeoutError` because no quorum existed. Twenty-eight causal reads reached `ExecutionTimeout` while waiting for an unavailable causal point. Twenty operations were

rejected before the wire by PyMongo `ConfigurationError` when a repeatedly restarted direct connection temporarily advertised no session support; these are client/topology availability errors and explain some unequal eligible denominators. This transient race is a limitation, but the remaining successful histories still directly expose rollback. H recorded 34/40 errors and no eligible property check, preferring unavailability to an acknowledged unsafe history.

5 Comparison with predictions

The observations agree with the documented boundaries and expose the purpose of each matched pair:

- A/B and C/D show that causal tracking prevents successful backward reads even with local read concern, although rollback can still invalidate a non-durable $w : 1$ predecessor.
- E/F isolates causal tracking with majority reads and $w : 1$. E returned stale committed snapshots; F waited or failed rather than cross its causal lower bound.
- G/H shows that majority concerns without causal tracking are not a substitute for a causal session. G violated RYW/MR under lag and 2+1 partition; H did not return a contradictory successful value.
- Comparing A/B/E/F ($w : 1$) with C/D/G/H (majority write) shows why acknowledgment durability matters during rollback. Majority-write histories became inconclusive when quorum was absent instead of accepting the isolated predecessor.

Zero observed violations are not proofs of universal guarantees. Guarantees come from the documented model; executions supply corroborating examples or counterexamples. Likewise, timeouts are evidence about availability rather than successful stale observations.

6 Limitations

1. All containers share one Mac and Docker's Linux VM, so host, kernel, disk, and power failures are correlated.
2. Direct connections intentionally force particular read targets. The experiment does not characterize normal driver load balancing.
3. A three-second delayed member is a laboratory control, not a realistic WAN model.
4. Each trial uses one document, a binary monotone version field, and sequential client operations. Cross-document causality, sharding, transactions, and concurrent overwrites are outside scope.
5. Fault scenarios use ten deterministic repetitions per configuration. This exposes timing variation but is still too small for production-frequency estimates.
6. Repeated restart cycles caused 20 PyMongo pre-command session-capability errors in rollback. They are reported as availability failures, but they reduced some eligible denominators and are a harness limitation.
7. The four property checks overlap within a history and must not be summed as statistically independent samples.
8. The durable-history treatment of rollback is explicit but not the only terminology used in MongoDB documentation.

7 Reproducibility and submitted evidence

The repository contains `compose.yaml`, `Dockerfile`, `lab.py`, `docker_backend.py`, `analyze.py`, checker unit tests, dependency pins, and a README. The formal A–H run is stored in this archive:

`matrix8-timeout5s-full-20260922-raw.zip`

- `history.jsonl`: operations, wire commands, returned values, causal tokens, fault events, topology snapshots, and recovery observations;
- `trials.json`: structured histories and four-property classifications;
- three `mongod-*.log` files containing elections, replication, and rollback;
- `metadata.json`, `audit.json`, and derived summaries.

`RAW_DATA_MANIFEST.json` supplies SHA-256 digests for every original file and the ZIP. Re-analysis must report 640 histories, 2720 attempted operations, 2700 wire commands, 20 pre-command errors, 160 isolation snapshots, and zero parameter, wire-coverage, or isolation audit failures.

8 AI usage disclosure

OpenAI Codex assisted with interpreting the assignment and MongoDB documentation, designing the experiment matrix, implementing and debugging the harness, executing local Docker tests, auditing raw evidence, and drafting this report. Database measurements were produced by real MongoDB processes. The group remains responsible for reviewing the code, validating the results, adding member information, and submitting the final work.

References

- [1] MongoDB, “Causal Consistency and Read and Write Concerns,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/causal-consistency-read-write-concerns/>
- [2] MongoDB, “Read Isolation, Consistency, and Recency,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/read-isolation-consistency-recency/>
- [3] MongoDB, “Write Concern,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/reference/write-concern/>
- [4] MongoDB, “Deploy a Self-Managed Replica Set for Testing and Development,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/tutorial/deploy-replica-set-for-testing/>
- [5] MongoDB, “Delayed Replica Set Members,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/replica-set-delayed-member/>
- [6] MongoDB, “Rollbacks During Replica Set Failover,” MongoDB 7.0 Manual. <https://www.mongodb.com/docs/v7.0/core/replica-set-rollbacks/>
- [7] MongoDB, “PyMongo ClientSession API,” PyMongo 4.10.1. https://pymongo.readthedocs.io/en/4.10.1/api/pymongo/client_session.html
- [8] Docker, “Compose file reference: services.” <https://docs.docker.com/reference/compose-file/services/>
- [9] DSA5208, “Lecture 3: Consistency Models,” course lecture slides, 2026.

A Exact configuration summary

MongoDB uses replica set `cc_lab`, a 128 MB oplog, and a 0.25 GB cache per node. Election/heartbeat intervals are 15000/500 ms. Read, write-concern, and socket deadlines are 5000/5000/7000 ms; retries are disabled and the random seed is 419. Recovery barriers use longer administrative timeouts than measured operations.